MASTER CAMPUS

App workflow, roles, database aur API guide

Is guide mein kya milega

Student, teacher aur campus admin ka workflow
Campus - session - branch - class hierarchy
Supabase login aur database requests ka flow
AI assistant, quiz generator aur file uploads
Setup dependencies aur practical limitations

Code walkthrough | 7 October 2026

Yeh document repository ke maujooda frontend, API code aur Supabase migrations ko dekhkar banaya gaya hai. Yeh live Supabase project ka health audit nahin hai. Kaunse features actual production mein chalenge, woh deployed Edge Functions, applied SQL migrations, RLS policies, Storage policies aur secrets par depend karta hai.

1. App ka overall architecture

Browser ya phone par React + Vite frontend chalta hai. User sign in ke baad Supabase Auth session paata hai. Frontend session token ke saath Supabase database/Storage ko directly call karta hai; row access ko database ke RLS policies control karte hain. AI requests ka current UI path Supabase Edge Function hai.

Phone / PC browser React + Vite UI	**Supabase Auth** user session + JWT	**Postgres + Storage** RLS protected data
Responsive web app / installable PWA shell	**campus-register** signup approval workflow	**campus-ai** AI assistant + quiz

Part	Role
React frontend	Login screens, dashboards, forms, attendance UI, upload controls, quiz display. Main file: src/main.jsx.
Supabase Auth	Email/password sign-in; JWT session; auth state restored on refresh. Client setup: src/supabase.js.
Supabase Postgres	Profiles, campuses, sessions, branches, classes, memberships, attendance, notices, assignments, reviews. RLS limits row access.
Supabase Storage	Private class-materials and assignment-submissions buckets; uploads and signed download links.
Supabase Edge Functions	campus-register creates approved-by-admin account requests; campus-ai calls Groq for AI work.
Node/Express API	server/index.js exposes /api/health, /api/assistant and /api/quiz on port 8787. Separate/duplicate AI backend path; current React UI invokes campus-ai Edge Function, not these Node AI routes.

Important: frontend ka live AI path
Ask Campus Buddy aur quiz generator `supabase.functions.invoke("campus-ai", ...)` chalate hain. Vite ka /api proxy Node server ko route karta hai, lekin current UI mein `/api/assistant` ya `/api/quiz` call nahin mila. Dono backends ko ek hi live path samajhna galat hoga.

2. Campus data structure

App ka intended flow ek campus ke andar academic session, session ke andar branch, aur branch/session ke andar class ya batch hai. Teacher branch-subject aur explicit class assignment ke through classes se judta hai. Students campus enrollment ke baad class membership request bhejte hain.

Level	Meaning / link
Campus	Campus admin registration par campus aur unique Campus ID banta hai.
Academic session	Campus ka session; jaise 2026-27.
Session + branch	`session_branches` batata hai kaunsi branch kis academic session mein active hai.
Branch + subject	Branch apne subjects rakhti hai; subject codes branch context mein define hote hain.
Class / batch	Class branch aur session se linked hoti hai; subjects `class_subjects` ke through link hote hain.
Teacher mapping	`branch_teachers` branch-subject level; `class_teachers` specific class level assignment. Ek teacher multiple classes/branches le sakta hai.
Student membership	`campus_enrollments` campus/branch/session affiliation; `class_memberships` specific class request/approval.

Campus admin ka initial setup

1. Admin account create/confirm karke login karta hai; campus record aur Campus ID milta hai.
2. Academic session banata hai, branches banata hai aur har branch ko session se link karta hai.
3. Branch subjects aur classes/batches define karta hai; class ke subjects select karta hai.
4. Teachers ki pending applications review/approve karta hai; branch-subject ya class teaching assignments karta hai.
5. Students Campus ID, session, branch aur college-issued registration number ke saath apply karte hain; admin campus application approve karta hai.
6. Student class code se class membership request bhejta hai; assigned teacher membership approve karta hai.

Current code mein class creation
`src/campusData.js` `create_campus_class` database RPC invoke karta hai. Latest relevant migration `20261014000000_fix_campus_admin_class_rls.sql` hai. Agar RPC/migration live database mein apply nahin hui, ya RLS helper ka role/campus mapping mismatch hai, class creation RLS error de sakta hai.

3. Login, registration aur approvals

Login UI role picker dikhata hai: Student, Teacher, Campus Admin; platform-admin path alag privileged use ke liye hai. Sign-in ke baad app `profiles` se role/status load karta hai aur selected login role actual profile role se match karta hai.

Action	Code path	Approval result
Login	Supabase `auth.signInWithPassword`; then `profiles` read.	Wrong role selected ho to sign out karke matching role choose karne ko kehta hai. Suspended account block hota hai.
Campus admin signup	Direct Supabase `auth.signUp`; metadata includes campus name/address.	DB trigger `handle_new_user` profile/campus/admin relationship create karta hai. Email confirmation Auth settings par depend karta hai.
Teacher signup	`campus-register` Edge Function; campus ID, email/password, profile metadata.	Function service-role Auth user create karta hai and pending campus enrollment/profile. Campus admin approval ke baad teacher workspace.
Student signup	`campus-register`; Campus ID, session, branch, college registration number.	Function campus and session-branch validate, duplicate college number check; campus admin approval. Class membership separately requested/teacher-approved.

College registration number aur app code

Student ka college-issued registration number user enter karta hai. Yeh campus ke andar duplicate nahin hona chahiye, aur roster attendance isi number ke mutabik sort hota hai. App ka generated internal student code alag identifier hai; dono ko confuse na karein. Registration uniqueness ke liye migrations `20261011000000_student_registration_candidate_key.sql` aur `20261012000000_college_registration_number.sql` current chain ka hissa hain.

Live setup check
Edge Functions ko deploy hona chahiye; campus-ai ke liye `GROQ_API_KEY` aur model secret set hona chahiye. SQL migrations ko correct order mein Supabase project par apply karna hoga. Frontend `.env` mein `VITE_SUPABASE_URL` aur `VITE_SUPABASE_ANON_KEY` public client config hain; service-role key kabhi Vite env/bundled frontend mein nahin rakhein.

4. Har role app mein kya karta hai

Navigation aur pages role ke hisab se change hote hain. Data calls current user/campus/class permissions se scope hone chahiye; dashboard counts aur screens tabhi real data dikhayenge jab Supabase config, database schema aur RLS sahi ho.

Role	Available workflow (current UI)
Campus admin	Overview; sessions; branches; teaching assignments; students/teachers approvals; routines; attendance; notices; calendar; activity log; support; classes. Teacher/student directory, approval, campus setup and exports.
Teacher	Overview; assigned classes; attendance roster; syllabus/material uploads; assignments; student reviews; notices; support. Can access student/class data only through assigned campus/class paths subject to RLS.
Student	Overview; my classes; attendance; assignments/submissions; study library/materials; calendar/notices; teacher reviews; help. Class join request by class code; study progress checkboxes.
Platform admin	Campus list/overview path intended for platform-level administration. This is separate from campus admin role and should be tightly controlled.

Class assignment vs branch assignment

Branch-subject assignment ka matlab teacher ko branch mein subject responsibility dena hai. Class ka access tabhi reliable hota hai jab `class_teachers` mein teacher-class row ho. Latest class RPC branch teachers ko selected class subjects ke mutabik auto-link kar sakta hai, lekin existing classes ke liye sync/assignment migration and UI action matter karte hain. Sirf branch linked dikhna, class assigned hone ka proof nahin hai.

Notice, assignment, material, progress

Teacher class ya branch context choose karke notice, assignment, syllabus unit aur learning material publish karta hai. Student enrolled class ki allowed resources dekhta hai. Study section ke checkbox progress ko save karte hain. Reviews/ratings role aur policy checks ke under submit aur read hote hain. File metadata database mein aur binary private Supabase Storage mein rehta hai.

5. API calls kaise chalti hain

Normal application requests Supabase JavaScript client se direct PostgREST/Storage/Auth endpoints ko jaati hain. Browser apna Supabase session JWT attach karta hai. Database RLS decide karta hai ki user ka SELECT/INSERT/UPDATE allowed hai ya nahin. Supabase ka anon/publishable key frontend identifier hai, authorization ka replacement nahin.

Feature	Frontend request	Backend action
Profile load	`from("profiles").select(...).eq("id", user.id)`	RLS ke mutabik apna/authorized profile return.
Campus signup	`auth.signUp(...)`	Supabase Auth + DB trigger; campus/profile/admin relation creation.
Teacher/student registration	`functions.invoke("campus-register", {body})`	Function campus code/session-branch validate karti hai; service-role Auth user create; profile/enrollment pending set.
Create class	`rpc("create_campus_class", {p_class_data, p_subject_ids})`	SQL function admin campus scope, active session-branch and subject validations; class, subjects, teacher links insert.
Read/write app records	`from(table).select/insert/update/upsert(...)`	Supabase PostgREST; RLS policy per operation/table.
Upload/download	Storage upload to private bucket; metadata in database; signed URL for read	Storage object policy and database policies both apply.
AI/quiz	`functions.invoke("campus-ai", {body})`	Edge Function verifies bearer user + profile/role/access; Groq API call; quiz result persisted.

Class creation API sequence

1. Admin form se branch, session, class name/code, subjects collect hote hain.

2. Frontend Supabase RPC `create_campus_class` call karti hai current auth token ke saath.

3. Postgres security-definer function caller ke campus-admin membership, session/branch relation, subject branch validate karta hai.

4. Valid hone par class insert, class-subject links aur applicable branch-teacher class links insert hote hain.

5. RLS/RPC error aaya to frontend ko error text milta hai; UI success tabhi dikhana chahiye jab RPC success ho.

6. Attendance aur files

Attendance teacher ke selected class aur date ke liye roster se start hoti hai. Roster active class members se load hota hai aur college registration number ke hisab se sort kiya jaata hai; absent number ho to app registration/internal code fallback ho sakta hai.

Attendance save sequence

1. Teacher class + date select karta hai; student status checkbox/selector se present/absent/late/excused mark karta hai.
2. Normal day par `attendance_sessions` class/date ke liye upsert hota hai, uske baad per-student `attendance_records` upsert.
3. Holiday/class suspended mark hone par session suspended save hota hai aur student absences create nahin hote.
4. Student/admin pages attendance history read karke totals/percentage dikhate hain; RLS unke allowed scope ko govern karta hai.

Attendance PDF ka actual behaviour
Teacher AI attendance review counts/percentages ka summary `campus-ai` ko bhejta hai. PDF export `window.print()` / browser Print > Save as PDF use karta hai. Current code mein AI-generated PDF file ka code nahin mila; AI review aur PDF printing alag features hain.

Upload aur student submissions

Frontend `src/fileRules.js` file type/size validate karta hai: PDF/CSV, max 10 MB; resource links HTTP/HTTPS hone chahiye. Teacher binary `class-materials` private bucket mein upload karta hai aur material/assignment/syllabus record database mein save karta hai. Student assignment file `assignment-submissions` bucket mein upload aur submission metadata database mein save hoti hai. Download ke liye signed URL milta hai.

Quiz ke uploaded PDF ka text extraction
Quiz generator binary PDF ko OCR/read nahin karta. Quiz source text database ke material/assignment/syllabus notes ya `content_text` description se aata hai. Isliye teacher ko useful lesson notes/text save karna hoga; sirf PDF upload se meaningful quiz guaranteed nahin.

7. AI assistant aur quiz generator

AI path UI se Supabase Edge Function `campus-ai` tak jaata hai, phir Groq API. Groq key client/browser mein nahin honi chahiye; Edge Function environment secret `GROQ_API_KEY` use karta hai. Model secret `GROQ_MODEL` current code/config mein `llama-3.3-70b-versatile` default hai.

AI action	What happens
Campus Buddy	UI current role/context ka message `action: assistant` ke saath Edge Function ko bhejta hai. Student message mein class count/attendance context add hota hai. Function user/session validate karke Groq response return karti hai.
Quiz generation	Student source list se material/assignment/syllabus source, count (5/10/15), difficulty choose karta hai. Frontend quiz action plus source kind/id invoke karta hai.
Authorization	Function JWT check karke active profile dekhti hai; quiz ke liye student role, source record aur active class membership validate karti hai.
Prompt input	Source ka saved text/notes use hote hain; kam text hua to request reject hoti hai. Current UI >30 chars guard; Edge Function source text >=40 char enforce karta hai.
Groq output	Function structured JSON multiple-choice questions generate karwati, option/index/explanation validate karti hai.
Persistence + score	Generated quiz database `generated_quizzes` mein save hota hai. Frontend answers score karta hai, completion score/time update karta hai.

AI request flow, step-by-step

1. Browser `supabase.functions.invoke` current user JWT ke saath request bhejta hai.

2. Edge Function token ko Supabase Auth se verify karke profile/account status aur action-specific access check karti hai.

3. Quiz action source record service-role client se read karta hai, par class membership/role checks ke baad hi content use karta hai.

4. Function GROQ_API_KEY ko server-side rakhti hai aur Groq chat completion endpoint ko prompt bhejti hai.

5. Response validate hota hai; quiz DB mein persist hota hai; answer UI par return hote hain.

Server API alag hai
Express server mein `/api/assistant` aur `/api/quiz` routes Groq call kar sakte hain. Vite dev server `/api` ko port 8787 par proxy karta hai, lekin current React UI ka observed AI call Supabase Edge Function hai. Node routes ko use karna ho to frontend ko explicitly un routes par point karna padega.

8. Setup, security aur known boundaries

Development mein root directory se `npm run dev` web + Node server start karta hai. Web `http://localhost:5173`; Node health/API port `8787`. Phone par same Wi-Fi test ke liye Vite network URL kholna, server origin/redirect settings sahi rakhna aur Supabase Auth redirect URLs mein intended origins allow karna zaruri hai. PWA install behavior browser/device ke hisab se badalta hai.

Config	Where	Purpose
VITE_SUPABASE_URL	Frontend `.env`	Supabase project URL; browser client.
VITE_SUPABASE_ANON_KEY	Frontend `.env`	Public anon/publishable key; keep RLS correct.
SUPABASE_URL	Node server env	Server-side Supabase project URL.
SUPABASE_SERVICE_ROLE_KEY	Node/Edge secret only	Privileged backend operations; never expose in Vite/browser.
GROQ_API_KEY	Edge Function / server secret	Groq API authentication; server-side only.
GROQ_MODEL	Edge Function / server env	Model selection.
APP_ORIGIN	Node server env	CORS allowed frontend origin.

What this code does not prove on its own

- Live deployment par har migration applied hai ya nahin. File name migration order batata hai, database state nahin.
- RLS aur Storage policies live project mein installed, correct, and secure hain ya nahin. A denied request may be a policy/membership mismatch; frontend code cannot override it.
- `campus-register`/`campus-ai` functions deployed hain, required secrets populated hain, and current production frontend points to the same project.
- Groq free quota, Supabase quotas/email limits, and production availability. Provider settings can change.
- AI factual accuracy. Teacher should review generated quizzes; student data shared in AI prompts should be minimized.

Useful troubleshooting clue
`Could not embed because more than one relationship...` usually means Supabase query needs an explicit foreign-key relation name. `new row violates row-level security policy` means caller/token/policy/campus membership/RPC path needs checking. `Edge Function returned a non-2xx` means inspect function logs/response body and deployed function secrets; the UI message alone does not identify the root cause.

Project locations

Frontend: `src/main.jsx`, `src/campusData.js`, `src/supabase.js`, `src/fileRules.js`
Functions: `supabase/functions/campus-register/index.ts`, `supabase/functions/campus-ai/index.ts`
Node API: `server/index.js`
Database migrations: `supabase/migrations/` (latest observed: `20261014000000_fix_campus_admin_class_rls.sql`)

End of guide